Projeto da 2ª Unidade - Implementação do K-means Utilizando MPI

Flávio
Graduando em Ciência da Computação
DIMAp - UFRN
Natal, Brasil
flavio.oliveira.124@ufrn.edu.br

Júlia Maria Azevedo Guilhermino
Graduanda em Ciência da Computação
DIMAp - UFRN
Natal, Brasil
juh.guilhermino03@gmail.com

Thiago Lopes Martins
Graduando em Ciência da Computação
DIMAp - UFRN
Natal, Brasil
thiago_lopes7@outlook.com

Kauã do Vale Ferreira
Graduando em Ciência da Computação
DIMAp - UFRN
Natal, Brasil
kaua.vale.632@ufrn.edu.br

***Abstract*—Este trabalho apresenta uma implementação de K-means em C++ com MPI aplicada ao Twitter Geospatial Data, com 14.262.517 registros. O agrupamento considera posição projetada e tempo, com normalização min-max. A leitura e a atribuição dos pontos são distribuídas entre processos, enquanto operações coletivas combinam as estatísticas de cada grupo. Três casos sintéticos foram validados com 1, 2 e 4 processos, com resultados compatíveis com os valores esperados nas nove execuções. No experimento com o dataset completo, o tempo mediano do K-means caiu de 40,96 s com um processo para 6,87 s com 16 processos, correspondendo a speedup de 5,96 e eficiência de aproximadamente 37%. A versão complementar CUDA apresentou 4,95 s na etapa de K-means. Os resultados indicam ganhos de desempenho no equipamento avaliado, com redução da eficiência nas maiores configurações.**

***Index Terms*—K-means, MPI, Programação Paralela, Clustering espaço-temporal**

I. Introdução

O objetivo deste trabalho é implementar o K-means com MPI e avaliar como o número de processos afeta seu tempo de execução. O algoritmo foi aplicado ao Twitter Geospatial Data, que reúne 14.262.517 tweets geolocalizados nos Estados Unidos, coletados entre 12 e 18 de janeiro de 2013 [1]. O agrupamento considera a posição e o horário de cada registro.

A implementação usa C++ e MPI (**Message Passing Interface**). O projeto também inclui uma versão CUDA e scripts Python para pré-processamento, análise exploratória e geração de gráficos. Os testes de desempenho foram executados em um único notebook, com diferentes quantidades de processos.

O pré-processamento converte longitude e latitude para metros em EPSG:5070 e transforma o timestamp em segundos relativos à referência 1970-01-01, mantendo a base temporal comum dos registros. Os horários originais estão em CST [1]. O atributo timezone permanece no CSV processado, mas não entra no agrupamento. As três dimensões utilizadas são x, y e tempo, normalizadas para o intervalo $[0, 1]$ e usadas no cálculo da distância sem pesos adicionais.

A Fig. 1 mostra a distribuição geográfica dos tweets na análise exploratória. Os grupos calculados pelo K-means são apresentados na seção de resultados.

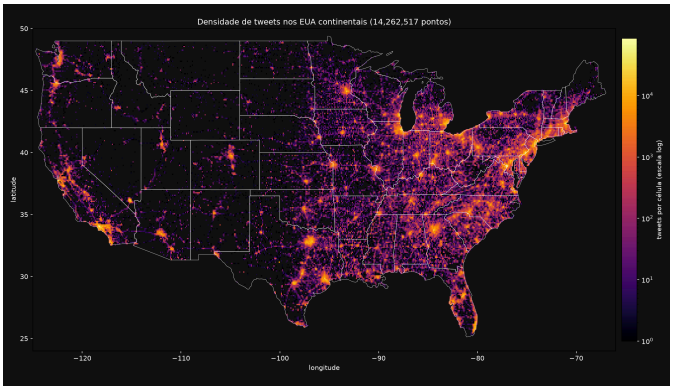


Fig. 1. Distribuição espacial dos tweets apresentada na análise exploratória.

II. Implementação

A. Leitura distribuída e normalização

Para lidar com os registros, adotou-se leitura paralela do dataset. Cada processo acessa o arquivo CSV e extrai a sua porção de dados, sem um processo central carregar e distribuir todo o conjunto. A divisão utiliza o tamanho do arquivo e o identificador de cada processo:

```
file.seekg(0, std::ios::end);
long long fileSize = file.tellg();
long long begin = fileSize * rank / size;
long long end = fileSize * (rank + 1) / size;
long long pos;

if (begin == 0) {
    // Igonra cabeçacho
    file.seekg(0);
    std::getline(file, line);
    pos = static_cast<long long>(line.size()) + 1;
} else {
    file.seekg(begin - 1);
    std::getline(file, line);
    pos = begin - 1 + static_cast<long long>(line.size()) + 1;
}
```

Fig. 2. Particionamento do arquivo entre os processos MPI.

Como uma fronteira pode cortar uma linha, a função `readData` usa `seekg` e `std::getline` para ajustar o início da leitura. O critério de particionamento é atribuir cada registro à faixa em que sua linha começa. Os dados locais são armazenados em um `std::vector<Tweet>`. Cada processo calcula seus mínimos e máximos, e a função `computeRange` utiliza `MPI_Allreduce` para obter os limites globais:

```
    Range range;
    MPI_Allreduce(lo.data(), range.min.data(), 3, MPI_DOUBLE, MPI_MIN, MPI_COMM_WORLD);
    MPI_Allreduce(hi.data(), range.max.data(), 3, MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD);
    return range;
}
```

Fig. 3. Reduções para obter os limites globais das dimensões.

Com os limites globais, cada processo normaliza seus próprios registros. Uma dimensão constante utiliza denominador 1, evitando divisão por zero. As reduções coletivas fazem parte do modelo de comunicação definido pelo padrão MPI [2].

B. Inicialização dos centroides

O K-means escolhe k índices globais distintos com `std::mt19937_64`, usando a semente informada na execução. As contagens locais e `MPI_Exscan` permitem identificar a posição de cada partição na sequência global dos registros. Apenas o processo que possui o ponto sorteado preenche suas coordenadas no buffer; os demais contribuem com zero. Uma redução por soma disponibiliza os centroides iniciais a todos:

```
std::vector<double> buf(3 * k, 0.0);
int c = 0;
for (long long g : chosen) {
    if (g >= offset && g < offset + static_cast<long long>(data.size())) {
        buf[3 * c] = data[g - offset].x;
        buf[3 * c + 1] = data[g - offset].y;
        buf[3 * c + 2] = data[g - offset].t;
    }
    ++c;
}
MPI_Allreduce(MPI_IN_PLACE, buf.data(), 3 * k, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
```

Fig. 4. Distribuição dos centroides iniciais por redução coletiva.

O mesmo arquivo e a mesma semente produzem os mesmos pontos iniciais. Durante as atualizações, porém, a ordem das somas em ponto flutuante pode variar com o número de processos e causar pequenas diferenças numéricas.

C. Atribuição, atualização e convergência

Em cada iteração, cada processo percorre seus pontos, calcula a distância euclidiana ao quadrado até os centroides e atribui cada ponto ao grupo mais próximo. Essa etapa é local. Depois, a redução abaixo verifica se alguma atribuição foi alterada:

```
int anyChanged;
MPI_Allreduce(&changed, &anyChanged, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);

if (!anyChanged && iter > 0) {
    if (rank == 0)
        std::cout << "Convergiu na iteracao "
                  << iter + 1 << '\n';
    break;
}
```

Fig. 5. Verificação global de alterações nas atribuições.

Quando nenhuma atribuição muda, o algoritmo informa convergência e encerra. Caso contrário, são calculadas somas e contagens locais por grupo. Duas chamadas adicionais combinam essas estatísticas:

```
MPI_Allreduce(MPI_IN_PLACE, sums.data(), 3 * k, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
MPI_Allreduce(MPI_IN_PLACE, counts.data(), k, MPI_LONG_LONG, MPI_SUM, MPI_COMM_WORLD);

for (int c = 0; c < k; ++c) {
    if (counts[c] > 0) {
        centroids[c].x = sums[3 * c] / counts[c];
        centroids[c].y = sums[3 * c + 1] / counts[c];
        centroids[c].t = sums[3 * c + 2] / counts[c];
    }
}
```

Fig. 6. Reduções das somas e contagens para atualizar os centroides.

Cada centroide é atualizado pela divisão entre soma global e contagem global. Clusters vazios mantêm a posição anterior. Uma iteração com atualização utiliza, portanto, três chamadas a `MPI_Allreduce`; a iteração que detecta convergência termina antes das duas reduções por soma. As mensagens dependem do número de clusters e dimensões, mas a latência e a sincronização das operações coletivas ainda podem limitar a escalabilidade.

D. Versão CUDA e instrumentação

A versão CUDA utiliza um kernel para atribuir os pontos e acumular estatísticas, com memória compartilhada e operações atômicas. As somas e contagens retornam à CPU, que atualiza os centroides a cada iteração. O modelo de programação CUDA é descrito pela NVIDIA [3].

O tempo medido no MPI é o maior entre os processos, e a leitura inclui a normalização. O intervalo do K-means MPI inclui a inicialização dos centroides. Na CUDA, essa inicialização ocorre antes do cronômetro do K-means; o intervalo cronometrado inclui as transferências das estatísticas por iteração e as atualizações na CPU. A transferência inicial dos dados para a GPU é medida à parte. Essas diferenças devem ser consideradas ao comparar as duas implementações.

TABLE III
MEDIANAS DE TRÊS REPETIÇÕES. TOTAL = LEITURA + K-MEANS, SEM A CÓPIA INICIAL PARA A GPU. FONTE: MEDICOES.CSV.

Configuração	Leitura (s)	K-means (s)	Total (s)	Speedup K-means	Eficiência MPI	Speedup total
MPI 1	3.89	40.96	44.87	1.00	100%	1.00
MPI 2	1.90	21.28	23.22	1.92	96%	1.93
MPI 4	1.00	10.98	11.99	3.73	93%	3.74
MPI 8	0.72	8.35	9.06	4.90	61%	4.95
MPI 16	0.46	6.87	7.33	5.96	37%	6.12
CUDA	3.84	4.95	8.79	8.28	-	5.10

III. TESTES E METODOLOGIA

A. Validação com dados sintéticos

Foram preparados três casos pequenos com resultados esperados calculáveis manualmente: média de quatro pontos, dimensão constante e dois grupos separados. Cada caso foi executado com 1, 2 e 4 processos, semente 42 e limite de 100 iterações, totalizando nove execuções. As contagens, centroides impressos e iterações coincidiram entre as configurações.

TABLE I
VALIDAÇÃO SINTÉTICA: RESULTADOS IGUAIS COM 1, 2 E 4 PROCESSOS.

Caso	Pontos	k	Iterações
Média	4	1	2
Dimensão constante	4	1	2
Dois grupos	6	2	3

O caso de média apresentou centroide (3,0; 6,0; 1,5), e o de dimensão constante apresentou (10,0; 5,0; 1,5). No caso de dois grupos, foram obtidos três pontos em cada grupo, com centroides (0,7; 0,7; 0,0) e (100,7; 100,7; 10,0). Nessas coordenadas, x e y estão em metros, e o tempo em horas após o início. Todas as execuções informaram convergência e não apresentaram coordenadas NaN ou infinitas na saída.

A comparação respeitou uma casa decimal, precisão impressa pelo programa. Os logs estão em `resultados/validacao_local/`, referentes ao commit `2031139`, com Open MPI 4.1.2 e g++ 11.4.0. Os tempos impressos como 0.000 s não foram usados na análise de desempenho.

A exportação foi verificada por cinco testes automatizados, executados com sucesso no notebook de Kauã. Eles cobrem execuções com e sem exportação, limite e repetibilidade da amostra, proteção contra sobrescrita, processos sem pontos locais e rejeição da visualização quando a execução não converge.

B. Experimento de desempenho

O ambiente e os parâmetros utilizados estão na Tabela II. Foram registradas três repetições para cada uma das cinco configurações MPI e três execuções CUDA, totalizando 18 medições no arquivo `medicoes.csv`. O limite de 300 iterações é um teto: o algoritmo pode convergir antes.

TABLE II
AMBIENTE E PARÂMETROS DO EXPERIMENTO DE DESEMPENHO.

Item	Configuração
CPU	Intel Core i5-13450HX; 6 P-cores + 4 E-cores; 16 threads
GPU	RTX 4050 Laptop; 5772 MB
Dataset	14.262.517 tweets
Parâmetros	k = 8; semente 42; limite 300
Processos	1, 2, 4, 8 e 16
Repetições	3 por configuração
Data	03/11/2026

Para cada configuração, foi calculada a mediana das três repetições. O speedup e a eficiência usam a própria implementação MPI com um processo como referência:

$$S(p) = \frac{T(1)}{T(p)} \quad E(p) = \frac{S(p)}{p} \tag{1}$$

O total considerado é leitura mais K-means, somados em cada repetição antes de calcular a mediana. Assim, ele pode diferir da soma das medianas das etapas. Não representa toda a pipeline nem inclui a transferência inicial para a GPU. O arquivo `resultados_cpu.md` apresenta médias; as tabelas e os gráficos deste relatório usam medianas de `medicoes.csv`.

IV. RESULTADOS E DISCUSSÃO

A. Desempenho

Os tempos e as métricas estão na Tabela III. O tempo mediano do K-means diminuiu de 40,96 s com um processo para 6,87 s com 16 processos. Até quatro processos, o ganho aproximou-se do crescimento proporcional, com speedup de 3,73 e eficiência de 93%. Com oito e 16 processos, os tempos continuaram caindo, mas os ganhos adicionais foram menores.

O processador combina núcleos de desempenhos diferentes e possui dez núcleos físicos. Com 16 processos, há compartilhamento desses núcleos. Isso e os custos de comunicação e sincronização podem contribuir para a queda de eficiência na Fig. 8. Os testes não mediram separadamente o efeito de cada fator.

A GPU apresentou 4,95 s na etapa de K-means, correspondendo a speedup de 8,28 em relação ao MPI com um processo. Entretanto, a soma medida de leitura e K-means foi menor no MPI com 16 processos: 7,33 s contra 8,79 s na CUDA. Portanto, a GPU foi mais rápida no cálculo, mas o MPI com 16 processos teve menor tempo na soma das etapas medidas. Os intervalos cronometrados diferem entre as versões, conforme descrito na seção de implementação.

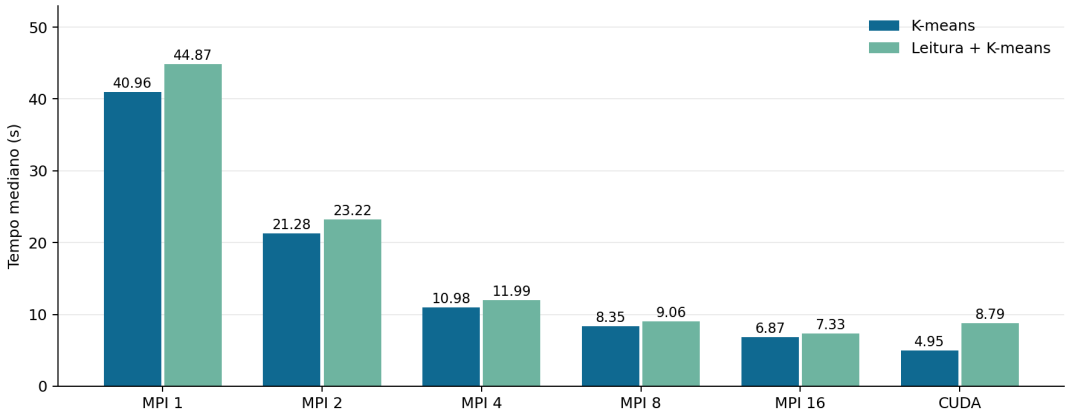


Fig. 7. Tempos medianos por configuração. As barras representam métricas distintas, não parcelas a somar.

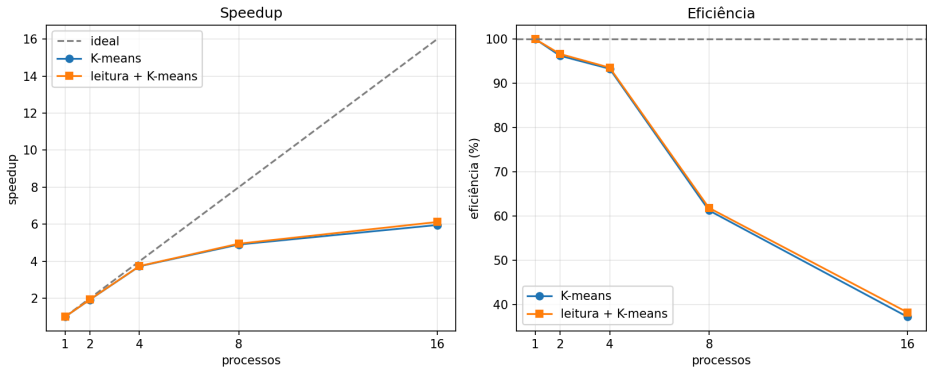


Fig. 8. Speedup e eficiência MPI, tendo um processo como referência.

O arquivo `medicoes.csv` registra os tempos, mas não o número de iterações de cada repetição. Os três tempos CUDA de `resultados_gpu.md` diferem dos registros do CSV e não foram combinados com eles. A origem dessa diferença ainda precisa ser esclarecida.

B. Resultados de clustering

Uma execução complementar no notebook de Kauã utilizou dois processos MPI, k = 8, semente 42 e limite de 300 iterações. Foram processados 14.262.517 registros, com convergência na iteração 262. As contagens e os centroides impressos coincidiram com os resultados CUDA de `resultados_gpu.md`, na precisão exibida. Não foi feita comparação ponto a ponto das atribuições entre as duas versões.

A execução registrou 10,221 s na leitura e 23,424 s no K-means e foi utilizada para exportar os rótulos e produzir a visualização. Esses tempos não integram as repetições da tabela de desempenho, realizadas em outro equipamento. O log está em `resultados/visualizacao_mpi_execucao.txt`.

A Tabela IV apresenta as contagens e os tempos dos centroides dessa execução. A soma dos oito grupos corresponde ao total de registros de entrada. Os IDs são apenas rótulos; o tempo de cada centroide é a média dos horários dos pontos do grupo, em horas após o início do conjunto.

TABLE IV
CONTAGENS E TEMPOS DOS CENTROIDES CONFIRMADOS NA EXECUÇÃO COMPLEMENTAR MPI.

Cluster	Tweets	t (h)
0	2.306.017	28.6
1	2.407.688	82.2
2	1.436.137	28.8
3	1.195.896	41.2
4	2.570.519	139.4
5	1.262.018	125.9
6	1.457.948	82.0
7	1.626.294	139.1

Os centroides dos IDs 0, 1 e 4; 2, 6 e 7; e 3 e 5 ocupam três regiões aproximadas do plano projetado, com separação temporal dentro desses conjuntos (Fig. 9). Assim, grupos espacialmente próximos podem se distinguir pelo horário dos registros. A figura mostra os centros; a distribuição dos pontos aparece na Fig. 10.

A Fig. 10 reúne 20.000 pontos com seus rótulos MPI, selecionados por amostragem uniforme dentro de cada partição, com limite de 10.000 pontos por processo. Os painéis superiores mostram a posição dos pontos e sua relação com o tempo. Os inferiores apresentam as contagens do conjunto completo e o tempo médio de cada cluster. A amostra serve à visualização e não deve ser usada diretamente para estimar a densidade global dos registros.

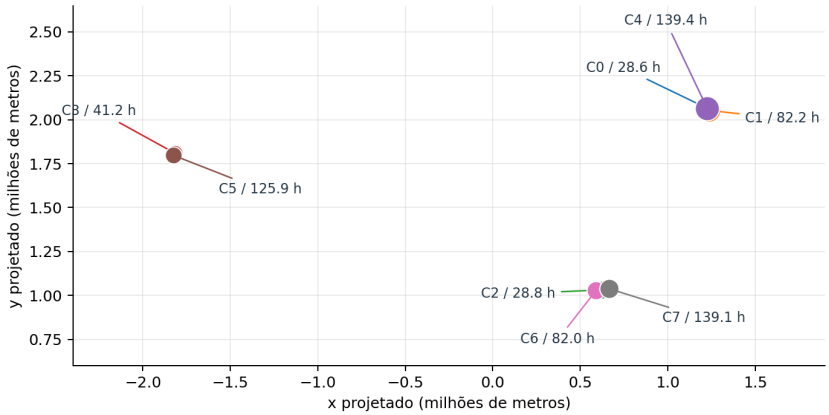


Fig. 9. Centroides da execução CUDA em EPSG:5070. A área dos círculos é proporcional à contagem; as anotações indicam ID e tempo. Centros próximos podem se sobrepor.

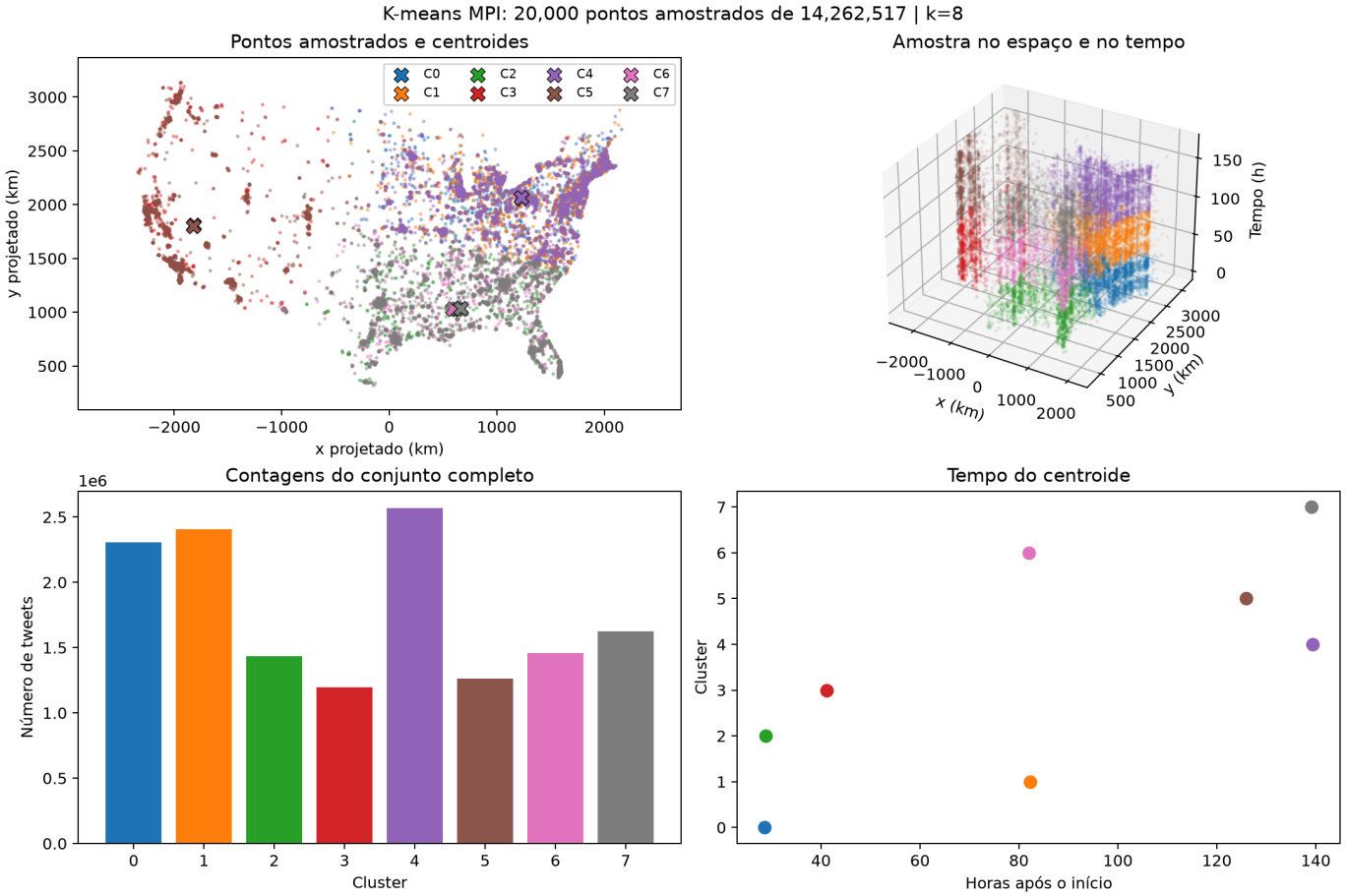


Fig. 10. Amostra de pontos com rótulos exportados pelo MPI, nos planos espacial e espaço-temporal. As barras apresentam as contagens do conjunto completo.

C. *Limitações*

O benchmark foi limitado a três repetições por configuração em um único notebook. Não foram avaliados outros equipamentos nem múltiplos nós. Os registros das medições originais não incluem o commit, o hash da entrada, as versões do compilador e do MPI ou os logs completos de cada repetição. Os arquivos da validação sintética e da execução para visualização documentam testes distintos.

A validação cobriu os casos descritos, mas ainda precisa abranger mais entradas inválidas, situações de leitura e clusters vazios. Não foram comparados outros valores de k nem calculadas métricas como silhueta. A normalização reduz o efeito das unidades, mas não garante influência idêntica das dimensões. A convergência também não demonstra que o agrupamento é ótimo ou que os grupos correspondem a cidades.

V. Conclusão

A implementação MPI distribuiu a leitura e o processamento dos pontos, combinando as estatísticas dos clusters por operações coletivas. Os três casos sintéticos produziram as contagens e os centroides esperados com 1, 2 e 4 processos. A execução do conjunto completo para visualização convergiu na iteração 262 e permitiu examinar os grupos no espaço e no tempo.

Nos testes de desempenho, o tempo mediano do K-means caiu de 40,96 s com um processo para 6,87 s com 16. O speedup chegou a 5,96, mas a eficiência diminuiu para 37%, com ganhos menores a partir de oito processos. A CUDA teve o menor tempo de cálculo; na soma de leitura e K-means, o MPI com 16 processos foi mais rápido nas condições avaliadas.

Os próximos passos são ampliar os casos de teste, completar os registros dos experimentos, uniformizar os intervalos de medição entre MPI e CUDA e avaliar outros valores de k e a qualidade dos agrupamentos.

References

[1] N. E. Helwig, Y. Gao, S. Wang, and P. Ma, "Twitter Geospatial Data." Accessed: Nov. 04, 2026. [Online]. Available: https://archive.ics.uci.edu/dataset/1050/twitter+geospatial+data

[2] MPI Forum, "MPI Documents." Accessed: Nov. 04, 2026. [Online]. Available: https://www.mpi-forum.org/docs/

[3] NVIDIA, "CUDA Programming Guide." Accessed: Nov. 04, 2026. [Online]. Available: https://docs.nvidia.com/cuda/cuda-programming-guide/index.html